

Codexrev CLI — Comprehensive Technical Documentation

Codexrev Research Documentation

August 2026

Executive Summary

- Key Highlights

- Project Metadata

Architecture Overview

- High-Level Architecture

- Module Dependency Graph

- Directory Structure

CLI Entry Point (`src/cli/index.ts`)

- Overview

- Command-Line Arguments

 - Global Options

 - Subcommands

 - Init Flags

- Execution Flow

- Code Structure

Configuration System (`src/config/`)

- Layered Settings Resolution

Default Settings

Settings Interface

Environment Variable Overrides

Provider API Key Environment Variables

Project Configuration

`.codexrev/config.json`

Settings Merge Algorithm

Core Agent Engine (`src/core/`)

Domain Types (`types.ts`)

Provider Identity

Content Parts

Tool Declaration

Stream Events

ContentGenerator Interface

Agent Turn Loop (`turn.ts`)

Agent Events

Agent Loop Algorithm

System Instruction

Tool Execution Routing

Multi-Agent Pipeline (`pipeline.ts`)

Three-Phase Architecture

Phase A: Committee (Read-Only Analysis)

Phase B: Breaker-Builder (Implementation)

Phase C: Resolver (Verification)

Pipeline Execution

Fix Loop (`fixLoop.ts`)

Algorithm

Retry Prompt

Interaction Modes (`modes.ts`)

Three Modes

Mode Cycling

Read-Only Tool Filter

Interaction Channel (`interaction.ts`)

- Pipeline Side (Producer)

- UI Side (Consumer)

Verification System (`verification.ts`)

- Three Strategies

- Test Runner Detection

Provider System (`src/providers/`)

- Provider Registry

- Metadata Table

- Provider Factory

- OpenAI-Compatible Base Class

- Streaming Implementation

- Message Conversion

- Anthropic Provider

- Google Gemini Provider

- Local Provider Health Check

Tool System (`src/tools/`)

- Tool Interface

- Built-in Tools

1. `read_file`
2. `write_file`
3. `edit`
4. `shell`
5. `glob`
6. `grep`
7. `web_fetch`
8. `web_search`
9. `ask_user` (Optional)

- Tool Registry

Sandbox System (`src/sandbox/`)

- Sandbox Architecture

Four Sandbox Backends

1. SeatbeltSandbox (macOS)
2. DockerSandbox
3. PodmanSandbox
4. NoSandbox (Off)

Resolution Strategy

Security System (`src/security/`)

Encryption Architecture

AES-256-GCM Encryption

Parameters

EncryptedPayload

Functions

OS Keychain Integration

Keytar Wrapper

Per-Project Key Binding

Keychain Operations

Test Mode

Error Hierarchy

MCP Integration (`src/mcp/`)

Model Context Protocol

Transport Types

Configuration

MCP Registry

Extension System (`src/extensions/`)

Extension Manifest

Extension Loading

Checkpointing System (`src/services/checkpoint.ts`)

Shadow Git Repository

Checkpoint Operations

Commit (Snapshot)

Restore

List

CheckpointRecord

Telemetry (`src/telemetry/`)

OpenTelemetry Integration

Functions

Configuration

Signal Handlers

Terminal UI (`src/ui/`)

React + Ink Architecture

Main Component (`App.tsx`)

Mode Badge

Clarification Prompt

Fix Confirm Prompt

Init Wizard (5 Steps)

Models Wizard (11+ Steps)

Public API (`src/api/index.ts`)

Exports

Usage Example

Build System

esbuild Configuration

Externals (Not Bundled)

Post-Build Steps

TypeScript Configuration

npm Scripts

Dockerfile

Test Suite

Test Files

Key Test Patterns

Keychain Tests (Fake Mode)

Secrets Round-Trip

Settings Merge

Error Handling

Error Hierarchy

Provider Error

Error Recovery

Configuration Files Reference

File Locations

Directory Permissions

Atomic Writes

Environment Variables Reference

Provider Keys

Settings Overrides

Internal

Appendix A: Provider Implementation Details

OpenAI Provider

Anthropic Provider

Google Gemini Provider

Ollama Provider

LM Studio Provider

LiteLLM Provider

Appendix B: Tool Parameter Schemas

read_file

write_file

edit

shell

glob

grep

web_fetch

web_search

Appendix C: Glossary

Appendix D: Version History

Roadmap Items

Executive Summary

Codexrev is a multi-provider, agentic command-line AI assistant built in TypeScript on Node.js 20+. It provides an intelligent coding assistant that can read, write, edit files, execute shell commands, fetch web content, and search the web — all orchestrated through a multi-agent pipeline with automatic verification and fix loops.

Key Highlights

- **Multi-Provider Support:** OpenAI, Anthropic, Google Gemini, Ollama, LM Studio, LiteLLM
- **Three-Phase Pipeline:** Committee (analysis) → Breaker-Builder (implementation) → Resolver (verification)
- **Fix Loop:** Automatic retry on verification failure with user confirmation
- **Three Modes:** Ask (read-only Q&A), Plan (analysis only), Agent (full pipeline)
- **Sandboxed Execution:** macOS Seatbelt, Docker, Podman, or passthrough
- **Encrypted API Key Storage:** AES-256-GCM with OS keychain-stored DEK
- **MCP Integration:** Model Context Protocol for external tool servers
- **React+Ink Terminal UI:** Interactive TUI with real-time streaming
- **Checkpointing:** Shadow git repos for point-in-time file snapshots
- **OpenTelemetry:** Built-in distributed tracing and metrics

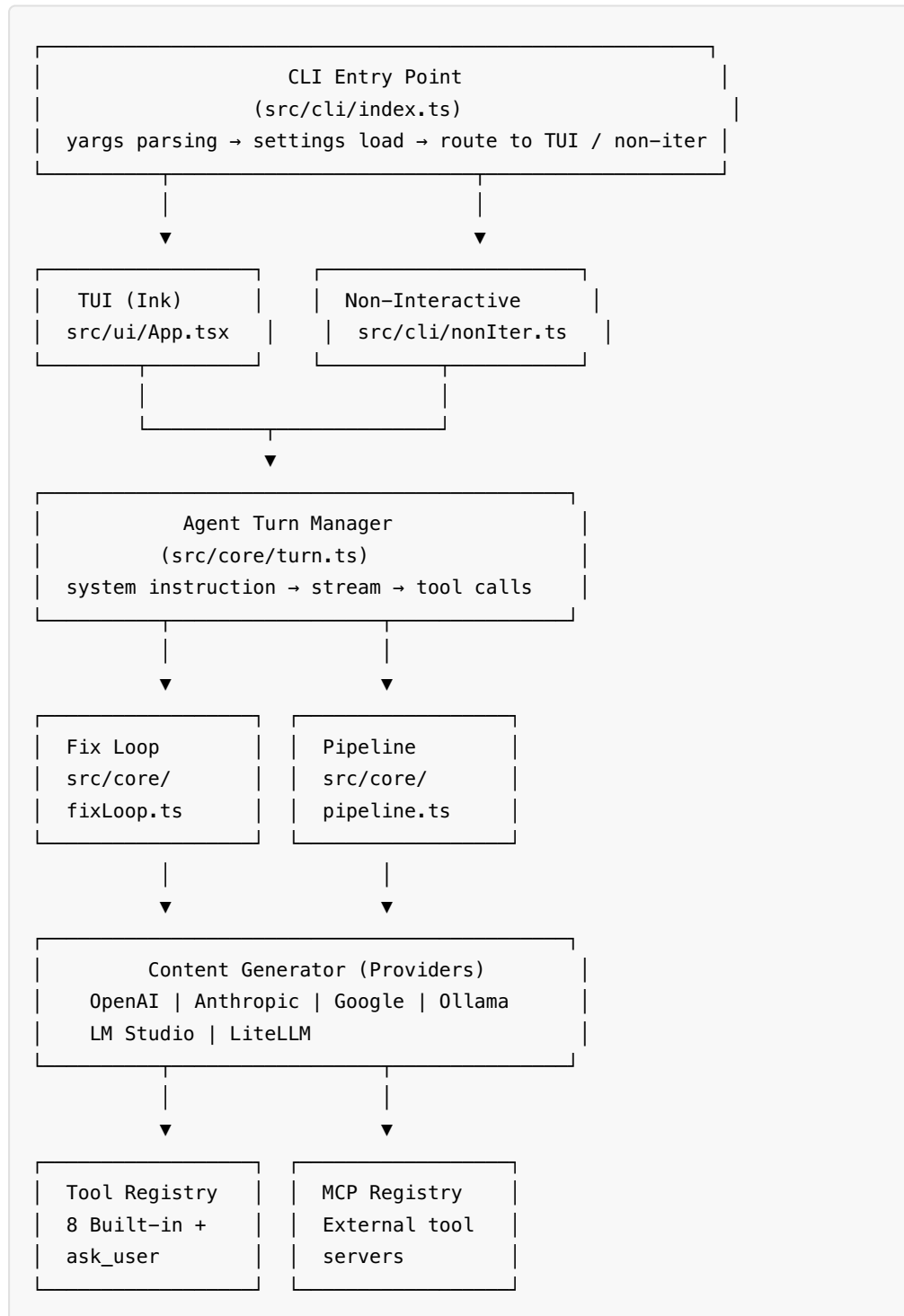
Project Metadata

Property	Value
Name	codexrev

Property	Value
Version	0.1.0
License	Apache-2.0
Node.js	>=20.0.0
Language	TypeScript (ES2023)
Module System	ESM ("type": "module")
Build Tool	esbuild
Test Framework	Vitest
Linters	ESLint
Formatter	Prettier

Architecture Overview

High-Level Architecture



Module Dependency Graph

```
cli/index.ts
├─ config/loader.ts      (settings resolution)
├─ config/models.ts      (multi-provider model registry)
├─ core/turn.ts          (agent loop)
│   └─ core/types.ts     (domain types)
│       └─ tools/registry.ts (tool dispatch)
│           └─ mcp/registry.ts (MCP tool dispatch)
├─ core/pipeline.ts      (3-phase pipeline)
├─ core/fixLoop.ts       (retry on failure)
├─ core/modes.ts         (ask/plan/agent)
├─ core/interaction.ts   (user Q&A channel)
├─ core/verification.ts  (test/LLM verification)
├─ providers/registry.ts (provider metadata)
├─ providers/index.ts    (provider factory)
├─ sandbox/manager.ts    (execution sandbox)
├─ security/keychain.ts  (DEK storage)
├─ security/secrets.ts   (AES-256-GCM)
├─ extensions/loader.ts  (plugin system)
├─ services/checkpoint.ts (shadow git)
├─ telemetry/index.ts    (OpenTelemetry)
└─ utils/*               (errors, logger, paths, env)
```

Directory Structure

```
codexrev/
├─ src/
│   └─ api/                Public programmatic API
│       └─ index.ts
│   └─ cli/                CLI entry points
│       ├── index.ts       Main entry (yargs)
│       ├── tui.ts         Interactive TUI launcher
│       ├── nonInteractive.ts --print mode
│       ├── init.ts        Project initialization
│       ├── help.ts        Help text rendering
│       ├── models.ts       Models subcommand
│       └─ modelsWizard.ts  Interactive model wizard
│   └─ config/             Configuration system
│       ├── index.ts       Re-exports
│       ├── loader.ts      Layered settings merge
│       ├── schema.ts      Settings types + defaults
│       ├── models.ts      Multi-provider CRUD
│       ├── projectConfig.ts .codexrev/config.json I/O
│       └─ projectSchema.ts Project config types
│   └─ core/               Agent engine
│       └─ types.ts        Domain types
```

```

├─ agent.ts      Programmatic agent factory
├─ turn.ts       Agent turn loop
├─ pipeline.ts   3-phase pipeline
├─ fixLoop.ts    Retry on failure
├─ modes.ts      Ask/Plan/Agent modes
├─ interaction.ts User Q&A channel
├─ verification.ts Test/LLM verification
├─ extensions/   Plugin system
│   └─ index.ts
│   └─ loader.ts  Extension loader
│   └─ types.ts   Manifest types
├─ mcp/          Model Context Protocol
│   └─ registry.ts MCP server management
├─ providers/    LLM provider adapters
│   └─ index.ts   Factory
│   └─ registry.ts Metadata registry
│   └─ _openaiCompat.ts Shared base class
│   └─ openai.ts  OpenAI adapter
│   └─ anthropic.ts Anthropic adapter
│   └─ google.ts   Google Gemini adapter
│   └─ ollama.ts   Ollama adapter
│   └─ lmstudio.ts LM Studio adapter
│   └─ litellm.ts  LiteLLM adapter
│   └─ health.ts   Local provider probe
├─ sandbox/      Execution sandbox
│   └─ index.ts
│   └─ manager.ts  4 sandbox backends
├─ security/      Encryption & keychain
│   └─ keychain.ts OS keychain wrapper
│   └─ secrets.ts  AES-256-GCM
├─ services/      Background services
│   └─ checkpoint.ts Shadow git
├─ telemetry/     OpenTelemetry
│   └─ index.ts
├─ tools/         Tool system
│   └─ index.ts
│   └─ registry.ts Tool registry
│   └─ specs.ts   Tool parameter schemas
│   └─ builtin.ts 8 built-in tools
│   └─ askUser.ts  Interactive clarification
├─ ui/            React+Ink TUI
│   └─ App.tsx     Main component
│   └─ ClarificationPrompt.tsx
│   └─ InitWizard.tsx
│   └─ ModeBadge.tsx
│   └─ ModelsWizard.tsx
├─ utils/         Shared utilities
│   └─ index.ts
│   └─ env.ts      Environment variables
│   └─ errors.ts   Error hierarchy
│   └─ logger.ts   Structured logging
│   └─ paths.ts    Path resolution

```

— tests/	Vitest test suite
— docs/	Documentation
— scripts/	Build scripts
— esbuild.config.mjs	Build configuration
— tsconfig.json	TypeScript config
— package.json	Package manifest

CLI Entry Point (`src/cli/index.ts`)

Overview

The CLI entry point is a ~300-line module that:

1. Parses command-line arguments using **yargs**
2. Loads layered settings via `loadSettings()`
3. Routes to subcommands (`init`, `extensions`, `models`)
4. Launches either the interactive TUI or non-interactive mode
5. Handles local provider health checks (Ollama, LM Studio, LiteLLM)

Command-Line Arguments

Global Options

Flag	Type	Description
<code>--provider</code>	string	LLM provider: <code>openai</code> , <code>anthropic</code> , <code>google</code> , <code>litellm</code> , <code>ollama</code> , <code>lmstudio</code>
<code>--model</code>	string	Model name (provider-specific)
<code>--sandbox</code>	string	Shell sandbox: <code>auto</code> , <code>seatbelt</code> , <code>docker</code> , <code>podman</code> , <code>off</code>
<code>--theme</code>	string	UI theme: <code>dark</code> , <code>light</code> , <code>solarized</code> , <code>monokai</code> , <code>nord</code>
<code>--telemetry</code>	boolean	Enable OpenTelemetry export
<code>--print</code> , <code>-p</code>	string	Run non-interactively with a prompt
<code>--output-format</code>	string	Output: <code>text</code> , <code>json</code> , <code>stream-json</code>
<code>--approval-mode</code>	string	Tool approval: <code>always</code> , <code>on-request</code> , <code>never</code>
<code>--debug</code>	boolean	Enable debug logging
<code>--no-update</code>	boolean	Skip update check

Subcommands

Command	Description
<code>codexrev init</code>	Initialize project with encrypted config
<code>codexrev extensions list</code>	List installed extensions
<code>codexrev extensions install <dir></code>	Install an extension
<code>codexrev extensions uninstall <name></code>	Remove an extension
<code>codexrev models list</code>	List all providers and models
<code>codexrev models add</code>	Add provider + model (interactive)
<code>codexrev models remove provider <name></code>	Remove a provider
<code>codexrev models remove model <id></code>	Remove a model
<code>codexrev models use <id></code>	Set active model

Init Flags

Flag	Type	Description
<code>--provider</code>	string	LLM provider
<code>--model</code>	string	Model name
<code>--api-key</code>	string	Provider API key
<code>--base-url</code>	string	Provider base URL
<code>--reset</code>	boolean	Replace existing config
<code>--non-interactive</code>	boolean	Skip the TUI wizard

Execution Flow

1. `ensureCodexrevHome()` → Create `~/.codexrev/` directories
2. `yargs parseAsync()` → Parse CLI arguments
3. Route subcommand → `init / extensions / models / main`
4. `loadSettings()` → Layered settings merge
5. Probe local providers → Health check for Ollama/LM Studio/LiteLLM
6. `loadExtensions()` → Scan `~/.codexrev/extensions/`
7. Route mode:
 - └─ `--print` → `runNonInteractive()`
 - └─ default → `runTui()`

Code Structure

```
// Simplified entry point flow
async function main(): Promise<void> {
  await ensureCodexrevHome();

  const argv = await yargs(hideBin(process.argv))
    .scriptName('codexrev')
    .command('init', 'Initialize this project', ...)
    .command('extensions', 'Manage installed extensions', ...)
    .command('models', 'Manage AI provider models', ...)
    .option('provider', { ... })
    .option('model', { ... })
    .option('sandbox', { ... })
    .option('theme', { ... })
    .option('print', { alias: 'p', ... })
    .parseAsync();

  // Subcommand routing
  if (argv._.includes('init')) { await runInit(flags); return; }
  if (argv._.includes('extensions')) { await
    handleExtensionsSubcommand(rest); return; }
  if (argv._.includes('models')) { await handleModelsCommand(sub, ...);
    return; }

  // Main flow
  const settings = await loadSettings();
  // Probe local providers, load extensions, then TUI or non-interactive
}
```

Configuration System (src/config/)

Layered Settings Resolution

The configuration system uses a **five-layer resolution** strategy with increasing priority:

```
Layer 1: DEFAULT_SETTINGS      (lowest priority)
Layer 2: ~/.codexrev/settings.json (user-level)
Layer 3: .codexrev/settings.json (project-level, legacy)
Layer 4: .codexrev/config.json  (project-level, encrypted)
Layer 5: Environment variables  (highest priority)
```

Each layer is deep-merged on top of the previous, with later layers overriding earlier ones.

Default Settings

```
const DEFAULT_SETTINGS: Settings = {
  provider: 'openai',
  model: 'gpt-4o',
  providers: {
    openai: { provider: 'openai', model: 'gpt-4o' },
    anthropic: { provider: 'anthropic', model: 'claude-3-5-sonnet-20241022' },
    google: { provider: 'google', model: 'gemini-1.5-pro' },
    litellm: { provider: 'litellm', model: 'gpt-4o' },
    ollama: { provider: 'ollama', model: 'llama3.1' },
    lmstudio: { provider: 'lmstudio', model: 'qwen2.5-7b-instruct' },
  },
  maxOutputTokens: 8192,
  temperature: 0.7,
  topP: 1.0,
  theme: 'dark',
  sandbox: 'auto',
  telemetry: false,
  checkpointing: true,
  mcpServers: {},
  toolTimeoutMs: 60_000,
```

```

maxTurns: 50,
approvalMode: 'on-request',
defaultMode: 'ask',
maxFixAttempts: 5,
verificationMode: 'auto',
metadata: {},
};

```

Settings Interface

```

interface Settings {
  provider: ProviderId;
  model: string;
  providers: Record<ProviderId, ProviderSettings>;
  maxOutputTokens: number;
  temperature: number;
  topP: number;
  theme: ThemeName;
  sandbox: SandboxMode;
  telemetry: boolean;
  checkpointing: boolean;
  mcpServers: Record<string, McpServerEntry>;
  toolTimeoutMs: number;
  maxTurns: number;
  approvalMode: 'always' | 'on-request' | 'never';
  defaultMode: InteractionMode;
  maxFixAttempts: number;
  verificationMode: VerificationMode;
  metadata: Record<string, unknown>;
}

```

Environment Variable Overrides

Environment Variable	Settings Key	Type
CODEXREV_PROVIDER	provider	ProviderId
CODEXREV_MODEL	model	string
CODEXREV_MAX_TOKENS	maxOutputTokens	number
CODEXREV_TEMPERATURE	temperature	number
CODEXREV_THEME	theme	ThemeName
CODEXREV_SANDBOX	sandbox	SandboxMode
CODEXREV_TELEMETRY	telemetry	boolean
CODEXREV_CHECKPOINTING	checkpointing	boolean

Environment Variable	Settings Key	Type
CODEXREV_DEFAULT_MODE	defaultMode	InteractionMode
CODEXREV_MAX_FIX_ATTEMPTS	maxFixAttempts	number
CODEXREV_VERIFICATION_MODE	verificationMode	VerificationMode
CODEXREV_LOG_LEVEL	(logger)	string
CODEXREV_OTEL_ENDPOINT	(telemetry)	string
CODEXREV_TEST_FAKE_KEYCHAIN	(keychain)	boolean

Provider API Key Environment Variables

Provider	API Key Env Var	Base URL Env Var
OpenAI	OPENAI_API_KEY	OPENAI_BASE_URL
Anthropic	ANTHROPIC_API_KEY	ANTHROPIC_BASE_URL
Google	GOOGLE_API_KEY	—
Ollama	OLLAMA_API_KEY	OLLAMA_BASE_URL
LM Studio	LMSTUDIO_API_KEY	LMSTUDIO_BASE_URL
LiteLLM	LITELLM_API_KEY	LITELLM_BASE_URL

Project Configuration

`.codexrev/config.json`

Encrypted project configuration stored in the project directory:

```
{
  "schemaVersion": 1,
  "provider": "openai",
  "model": "gpt-4o",
  "apiKey": {
    "iv": "base64-encoded-iv",
    "tag": "base64-encoded-gcm-tag",
    "ciphertext": "base64-encoded-ciphertext"
  },
  "providers": [
    {
      "name": "openai",
      "vendor": "openai",
      "baseUrl": "https://api.openai.com/v1",
      "apiKey": { "iv": "...", "tag": "...", "ciphertext": "..." },
      "models": [
```

```

    {
      "id": "gpt-4o",
      "name": "GPT-4o",
      "url": "https://api.openai.com/v1",
      "toolCalling": true,
      "vision": true,
      "maxInputTokens": 128000,
      "maxOutputTokens": 16384,
      "default": true
    }
  ]
}

```

Settings Merge Algorithm

```

async function loadSettings(cwd: string = process.cwd()):
  Promise<Settings> {
    let merged: Settings = DEFAULT_SETTINGS;

    // 1) User-level settings
    const userRaw = await readJsonFile(getCodexrevPaths().settingsFile);
    merged = deepMerge(merged, userRaw);

    // 2) Project-level settings (legacy)
    const projectPath = await findProjectConfig(cwd);
    merged = deepMerge(merged, await readJsonFile(projectPath));

    // 2b) Encrypted project config
    merged = await mergeProjectConfig(merged, cwd);

    // 3) Env overrides
    merged = applyEnvOverrides(merged);
    merged = ensureProviderKeys(merged);

    return merged;
  }

```

Core Agent Engine (src/core/)

Domain Types (types.ts)

Provider Identity

```
type ProviderId = 'openai' | 'anthropic' | 'google' | 'litellm' |
  'ollama' | 'lmstudio';
type Role = 'system' | 'user' | 'assistant' | 'tool';
```

Content Parts

The system uses a **discriminated union** for content parts:

```
interface TextPart    { kind: 'text'; text: string; }
interface BlobPart    { kind: 'blob'; mimeType: string; data: string; /*
  base64 */ }
interface ToolCallPart { kind: 'tool_call'; id: string; name: string;
  arguments: unknown; }
interface ToolResultPart { kind: 'tool_result'; toolCallId: string;
  name: string;
                        content: Array<TextPart | BlobPart>;
  isError?: boolean; }

type ContentPart = TextPart | BlobPart | ToolCallPart | ToolResultPart;
interface Message { role: Role; parts: ContentPart[]; }
```

Tool Declaration

```
interface ToolDeclaration {
  name: string;
  description: string;
  parameters: {
    type: 'object';
    properties: Record<string, unknown>;
    required?: readonly string[];
    additionalProperties?: boolean;
  };
}
```

Stream Events

```
type StreamEvent =
  | { kind: 'message_start'; messageId: string }
  | { kind: 'text_delta'; text: string }
  | { kind: 'tool_call'; toolCall: ToolCallPart }
  | { kind: 'finish'; reason: 'stop' | 'max_tokens' | 'tool_use' |
      'error' | 'aborted';
      usage?: UsageStats }
  | { kind: 'error'; error: ProviderError };
```

ContentGenerator Interface

Every LLM provider must implement this interface:

```
interface ContentGenerator {
  readonly provider: ProviderId;
  readonly capabilities: ProviderCapabilities;
  generate(req: GenerateRequest): Promise<GenerateResponse>;
  stream(req: GenerateRequest): AsyncIterable<StreamEvent>;
}

interface ProviderCapabilities {
  readonly supportsTools: boolean;
  readonly supportsStreamingUsage: boolean;
}
```

Agent Turn Loop (`turn.ts`)

The agent turn is the core execution unit — a single user → model → tool(s) → model exchange.

Agent Events

```
type AgentEvent =
  | { kind: 'message_start' }
  | { kind: 'text_delta'; text: string }
  | { kind: 'tool_call'; toolCall: ToolCallPart }
  | { kind: 'tool_result'; toolResult: ToolResultPart }
  | { kind: 'usage'; usage: UsageStats }
  | { kind: 'turn_complete'; turns: number; reason: string }
  | { kind: 'error'; error: ProviderError }
  | { kind: 'pipeline_phase'; phase: 'committee' | 'breaker-builder' |
      'resolver' };
```

```

    description: string }
| { kind: 'fix_iteration'; attempt: number; maxAttempts: number;
    status: 'red' | 'green' | 'checking'; summary: string }
| { kind: 'clarification_prompt'; question: string; suggestions:
    readonly string[] }
| { kind: 'clarification_response'; answer: string };

```

Agent Loop Algorithm

```

streamAgent(opts):
    messages = [contextMessages..., userPrompt]
    systemInstruction = buildSystemInstruction(settings, suffix)
    toolDecls = builtinTools + mcpTools

    for turn in 0..maxTurns:
        if signal.aborted → yield turn_complete(aborted)

        yield message_start
        text = ""
        toolCalls = []

        for event in provider.stream(model, messages, tools,
systemInstruction):
            if text_delta → yield text_delta, accumulate text
            if tool_call → yield tool_call, collect call
            if finish      → yield usage
            if error       → yield error, return

        messages.append(assistantMessage)

        if no toolCalls → yield turn_complete, return

        for call in toolCalls:
            result = executeToolCall(call, tools, mcp)
            messages.append(toolResult)
            yield tool_result

    yield turn_complete(max_turns)

```

System Instruction

You are Codexrev, a multi-provider agentic CLI assistant.
 Working directory: {cwd}
 Provider: {provider}
 Model: {model}
 When you need to use tools, prefer the most specific tool.
 If the user has not yet approved a destructive action, ask for

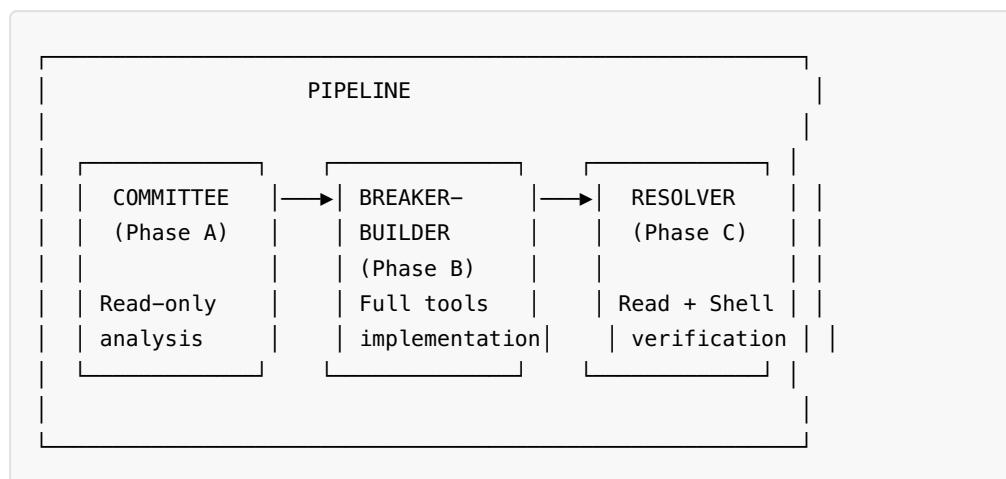
```
confirmation.
{modeSuffix}
```

Tool Execution Routing

```
async function executeToolCall(call, tools, mcp, ctx):
    Promise<ToolResult> {
    const builtin = tools.get(call.name);
    if (builtin) return await builtin.execute(call.arguments, ctx);
    return await mcp.callTool(call.name, call.arguments);
    }
```

Multi-Agent Pipeline (pipeline.ts)

Three-Phase Architecture



Phase A: Committee (Read-Only Analysis)

Purpose: Analyze the user's request and produce a structured assessment.

Available Tools: `read_file`, `glob`, `grep`, `web_fetch`, `web_search`, `ask_user`

Output Format:

1. **Problem Summary** — what the user wants
2. **Affected Files** — files to examine or modify
3. **Proposed Approach** — step-by-step plan

4. Risk Assessment — what could go wrong

Phase B: Breaker-Builder (Implementation)

Purpose: Implement the changes planned by the Committee.

Available Tools: All 8 built-in tools + `ask_user`

Input: Committee's analysis + original prompt

Behavior: - Follow the Committee's plan step by step - Use `edit / write_file` for code changes - Use `shell` for build/compile commands - Prefer minimal, targeted edits over large rewrites

Phase C: Resolver (Verification)

Purpose: Verify that changes are correct.

Available Tools: `read_file`, `glob`, `grep`, `web_fetch`, `web_search`, `shell`, `ask_user`

Output: Must start with `GREEN:` (success) or `RED:` (failure) followed by explanation.

Pipeline Execution

```
async function* runPipeline(opts): AsyncGenerator<AgentEvent,
    PipelineResult> {
    // Phase A: Committee
    const analysis = yield* runCommittee(opts);

    // Phase B: Breaker-Builder
    const implementation = yield* runBreakerBuilder(opts, analysis);

    // Phase C: Resolver
    const verification = yield* runResolver(opts, opts.prompt,
        implementation);

    const passed = /^s*GREEN:/im.test(verification);

    return { committeeAnalysis: analysis, implementation, verification,
        passed };
}
```

Fix Loop (`fixLoop.ts`)

Algorithm

```
runFixLoop(opts):
  attempts = 0
  maxAttempts = opts.maxFixAttempts ?? 5

  // First pass: full pipeline
  attempts = 1
  lastResult = runPipeline(opts)

  if lastResult.passed → return { passed: true, attempts: 1 }

  // Retry loop
  while attempts < maxAttempts:
    decision = interactionChannel.requestFixConfirmation(attempts,
maxAttempts, ...)
    if decision === 'stop' → return { stoppedByUser: true }

    attempts++
    retryResult = runRetryIteration(opts, previousErrors)
    // (skips Committee, re-runs Breaker-Builder + Resolver)

    if retryResult.passed → return { passed: true, attempts }

    previousErrors = retryResult.verification

  return { passed: false, attempts, lastResult }
```

Retry Prompt

You are the BREAKER-BUILDER agent – retrying a fix after verification failure.

The previous attempt was marked RED by the Resolver.

Instructions:

1. Read the Resolver's feedback carefully.
2. Examine the current state of the affected files.
3. Make targeted fixes to address the issues identified.
4. Do not redo work that was already correct.

Interaction Modes (`modes.ts`)

Three Modes

Mode	Label	Color	Read-Only	Description
<code>ask</code>	Ask	Blue	Yes	Read-only Q&A — no file edits, no shell commands
<code>plan</code>	Plan	Yellow	Yes	Generates step-by-step plan — no execution
<code>agent</code>	Agent	Green	No	Full pipeline with fix loop

Mode Cycling

```
function nextMode(current: InteractionMode): InteractionMode {
  const idx = MODE_ORDER.indexOf(current);
  return MODE_ORDER[(idx + 1) % MODE_ORDER.length];
}
// Ask → Plan → Agent → Ask → ...
```

Read-Only Tool Filter

```
const READ_ONLY = new Set(['read_file', 'glob', 'grep', 'web_fetch',
  'web_search', 'ask_user']);
```

Interaction Channel (`interaction.ts`)

The `InteractionChannel` is an **EventEmitter-based async producer-consumer** mechanism for mid-run pauses.

Pipeline Side (Producer)

```
// Blocks until UI responds
const answer = await interactionChannel.requestClarification(
  'Which approach do you prefer?',
  ['Option A', 'Option B']
);
```

```
const decision = await interactionChannel.requestFixConfirmation(
  attempt, maxAttempts, summary, 'red'
);
// Returns 'continue' or 'stop'
```

UI Side (Consumer)

```
// Subscribe to events
interactionChannel.onInteraction((event) => {
  if (event.type === 'clarification_request') {
    // Show question to user
    interactionChannel.respondClarification(event.request.id,
      userAnswer);
  }
  if (event.type === 'fix_confirm_request') {
    // Show continue/stop to user
    interactionChannel.respondFixConfirmation(event.request.id,
      decision);
  }
});
```

Verification System (verification.ts)

Three Strategies

Strategy	Behavior
tests	Auto-detects test runner, runs tests, checks exit code
llm	Asks the model to evaluate with GREEN: / RED: prefix
auto	Tries tests first, falls back to LLM

Test Runner Detection

```
const checks = [
  { file: 'package.json', cmd: '' }, // Parse scripts.test
  { file: 'pytest.ini', cmd: 'pytest' },
  { file: 'pyproject.toml', cmd: 'pytest' },
  { file: 'setup.cfg', cmd: 'pytest' },
  { file: 'go.mod', cmd: 'go test ./...' },
  { file: 'Cargo.toml', cmd: 'cargo test' },
  { file: 'Makefile', cmd: '' }, // Check for test: target
  { file: 'Gemfile', cmd: 'bundle exec rspec' },
];
```

Provider System (src/providers/)

Provider Registry

Metadata Table

Provider	Label	Default Model	Default Base URL	Requires Key	Env Key Var	Support Tool
openai	OpenAI	gpt-4o	https://api.openai.com/v1	Yes	OPENAI_API_KEY	Yes
anthropic	Anthropic	claude-3-5-sonnet-20241022	https://api.anthropic.com	Yes	ANTHROPIC_API_KEY	Yes
google	Google Gemini	gemini-1.5-pro	—	Yes	GOOGLE_API_KEY	Yes
litellm	LiteLLM	gpt-4o	http://localhost:4000	No	LITELLM_API_KEY	Yes
ollama	Ollama	llama3.1	http://localhost:11434/v1	No	OLLAMA_API_KEY	Yes
lmstudio	LM Studio	qwen2.5-7b-instruct	http://localhost:1234/v1	No	LMSTUDIO_API_KEY	Yes

Provider Factory

```
function buildProvider(settings: Settings): ContentGenerator {
  const { provider, model } = settings;
  const providerSettings = settings.providers[provider];

  // Resolve API key: settings → env var
  const apiKey = providerSettings?.apiKey ??
    process.env[providerMeta(provider).envKeyVar];
  const baseUrl = providerSettings?.baseUrl ??
    providerMeta(provider).defaultBaseUrl;

  assertKey(provider, apiKey);

  switch (provider) {
    case 'openai': return new OpenAIGenerator({ model, apiKey,
      baseUrl, ... });
```

```

    case 'anthropic': return new AnthropicGenerator({ model, apiKey,
        baseUrl, ... });
    case 'google':    return new GoogleGenerator({ model, apiKey, ... });
    case 'ollama':    return new OllamaGenerator({ model,
        baseUrl, ... });
    case 'lmstudio':  return new LMStudioGenerator({ model,
        baseUrl, ... });
    case 'litellm':   return new LiteLLMGenerator({ model, apiKey,
        baseUrl, ... });
  }
}

```

OpenAI-Compatible Base Class

Four providers share a common base: `openai`, `ollama`, `lmstudio`, `litellm`.

Streaming Implementation

1. Build OpenAI API params from Message[]
2. POST to /chat/completions with stream=true
3. Parse NDJSON chunks
4. Accumulate tool calls across chunks
5. Map finish_reason to StreamEvent

Message Conversion

```

function toApiParams(messages: Message[], tools: ToolDeclaration[]):
  OpenAIParams {
  return {
    model, temperature, topP, maxTokens,
    messages: messages.map(m => ({
      role: m.role,
      content: m.parts.filter(p => p.kind === 'text').map(p =>
        p.text).join(''),
      tool_calls: m.parts.filter(p => p.kind === 'tool_call').map(tc =>
        ({
          id: tc.id, type: 'function',
          function: { name: tc.name, arguments:
            JSON.stringify(tc.arguments) },
        })),
      tool_call_id: m.parts.find(p => p.kind ===
        'tool_result')?.toolCallId,
    })),
    tools: tools.map(t => ({
      type: 'function',
      function: { name: t.name, description: t.description, parameters:
        t.parameters },
    })),
  }
}

```

```
};
}
```

Anthropic Provider

Uses `@anthropic-ai/sdk` with custom streaming:

- `content_block_start` → new content block
- `delta` → `text` or `input_json_delta` for tool calls
- `stop` → finish with usage
- Reports `cache_read` / `cache_write` tokens

Google Gemini Provider

Uses `@google/genai`:

- Converts to Google's `Content` format
- Tool calls via `functionCall` parts
- Supports `thinkingConfig` for extended thinking

Local Provider Health Check

```
async function probeLocalProvider(provider: string, baseUrl: string):
  Promise<void> {
  const controller = new AbortController();
  const timeout = setTimeout(() => controller.abort(), 5_000);

  try {
    const res = await fetch(`${baseUrl}/models`, {
      signal: controller.signal,
      headers: { 'Accept': 'application/json' },
    });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
  } catch (err) {
    throw new Error(
      `Cannot reach ${provider} at ${baseUrl}. ` +
      `Make sure the ${provider} daemon is running.`
    );
  } finally {
    clearTimeout(timeout);
  }
}
```

```
}  
}
```

Tool System (src/tools/)

Tool Interface

```
interface Tool {
  name: string;
  description: string;
  parameters: ToolParameters;
  declaration: ToolDeclaration;
  execute(args: unknown, ctx: ToolContext): Promise<ToolResult>;
}

interface ToolContext {
  cwd: string;
  signal?: AbortSignal;
  ask?: (toolName: string, args: unknown) => Promise<boolean>;
}

interface ToolResult {
  output: string | unknown;
  isError?: boolean;
}
```

Built-in Tools

1. read_file

Reads file contents with optional offset/limit.

Parameters:

Name	Type	Required	Description
file_path	string	Yes	Path to the file (absolute or relative to cwd)
offset	number	No	Starting line number (0-based)
limit	number	No	Number of lines to read

2. `write_file`

Creates directories and writes file content.

Parameters:

Name	Type	Required	Description
<code>file_path</code>	<code>string</code>	Yes	Path to the file
<code>content</code>	<code>string</code>	Yes	Content to write

3. `edit`

String replacement in files.

Parameters:

Name	Type	Required	Description
<code>file_path</code>	<code>string</code>	Yes	Path to the file
<code>old_string</code>	<code>string</code>	Yes	Text to find and replace
<code>new_string</code>	<code>string</code>	Yes	Replacement text
<code>replace_all</code>	<code>boolean</code>	No	Replace all occurrences (default: first only)

4. `shell`

Execute shell commands with timeout and approval.

Parameters:

Name	Type	Required	Description
<code>command</code>	<code>string</code>	Yes	Shell command to execute
<code>timeout</code>	<code>number</code>	No	Timeout in ms (default: 30,000)

Behavior: - Uses `/bin/sh -c` on Unix, `cmd.exe /c` on Windows -
 Max buffer: 8 MB - Timeout: 30s default - Approval callback via
`ctx.ask()` when configured - Captures both stdout and stderr

5. `glob`

File pattern matching using the `glob` npm package.

Parameters:

Name	Type	Required	Description
pattern	string	Yes	Glob pattern
cwd	string	No	Working directory

6. grep

Regex search across files.

Parameters:

Name	Type	Required	Description
pattern	string	Yes	Regex pattern
path	string	Yes	Directory to search
include	string	No	File glob filter (default: <code>**/*</code>)
case_sensitive	boolean	No	Case sensitivity

Output: Up to 500 matches in format `file:line:content`

7. web_fetch

Fetch and convert web page content to text.

Parameters:

Name	Type	Required	Description
url	string	Yes	HTTP/HTTPS URL

Behavior: - Uses `fetch()` with redirect following - Converts HTML to text using `html-to-text` - Strips links - Output truncated to 20,000 characters

8. web_search

DuckDuckGo HTML scraping for web search.

Parameters:

Name	Type	Required	Description
query	string	Yes	Search query

Name	Type	Required	Description
num_results	number	No	Number of results

Behavior: - Uses DuckDuckGo HTML endpoint (no API key needed) -
Output truncated to 8,000 characters

9. ask_user (Optional)

Interactive clarification tool bound to the InteractionChannel.

Parameters:

Name	Type	Required	Description
question	string	Yes	Question to ask the user
suggestions	string[]	No	Predefined answer options

Tool Registry

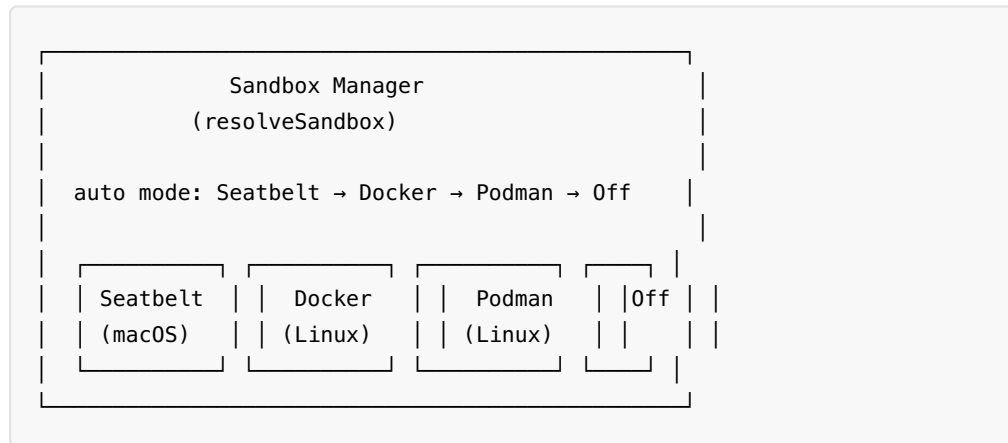
```
class ToolRegistry {
  private tools = new Map<string, Tool>();

  register(tool: Tool): void;
  unregister(name: string): boolean;
  get(name: string): Tool | undefined;
  list(): Tool[];
  asMap(): Map<string, Tool>;
}

function createToolRegistry(settings: Settings, askTool?: Tool):
  ToolRegistry {
  const reg = new ToolRegistry();
  for (const tool of builtinTools(settings)) {
    reg.register(tool);
  }
  if (askTool) reg.register(askTool);
  return reg;
}
```

Sandbox System (src/sandbox/)

Sandbox Architecture



Four Sandbox Backends

1. SeatbeltSandbox (macOS)

Uses macOS `sandbox-exec` with SBPL profiles.

Profile:

```
(version 1)
(deny default)
(allow process-exec)
(allow file-read*)
(allow sysctl-read)
(allow mach-lookup)
(allow network*)
(allow file-write* (subpath "/dev/null"))
(allow file-write* (subpath "/tmp"))
(allow file-write* (subpath "/private/tmp"))
(allow file-write* (subpath "{cwd}"))
```

Behavior: - Denies all operations by default - Allows file reads everywhere - Allows writes only to `/tmp`, `/dev/null`, and the working directory - Profile written to temp file, cleaned up after execution

2. DockerSandbox

Runs commands inside `docker run --rm` containers.

```
const args = [
  'run', '--rm', '-i',
  '-v', `${cwd}:/work`,
  '-w', '/work',
  ...envVars,
  'alpine:3.20',
  ...argv,
];
```

Behavior: - Uses `alpine:3.20` image - Mounts working directory to `/work` - Passes environment variables via `-e` - Default timeout: 120 seconds

3. PodmanSandbox

Identical to Docker but uses the `podman` CLI.

4. NoSandbox (Off)

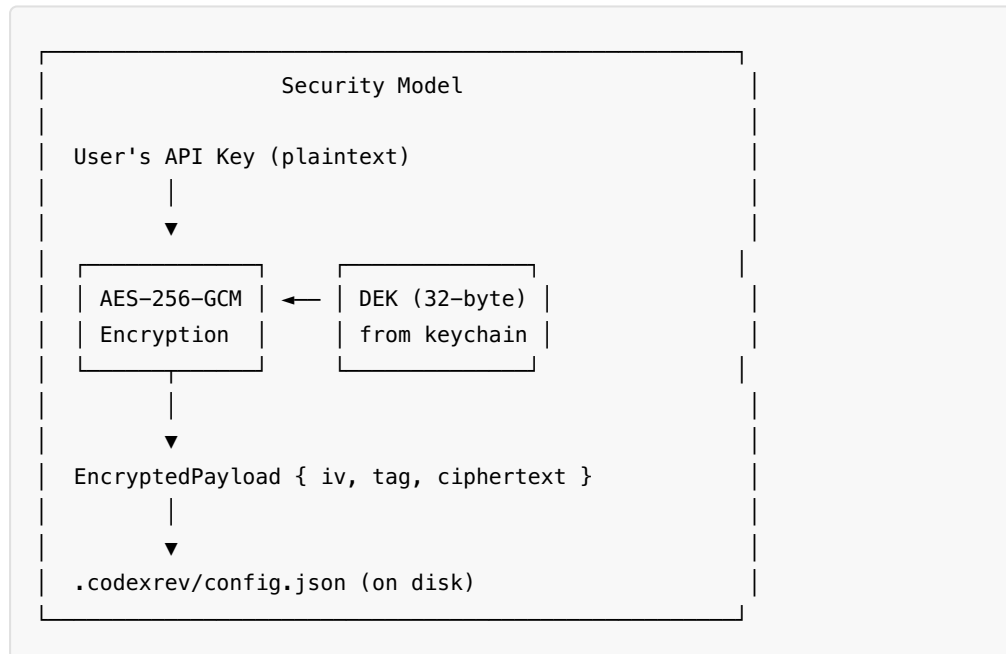
Direct `execFile` passthrough — no isolation.

Resolution Strategy

```
async function resolveSandbox(mode: SandboxMode = 'auto'):
  Promise<ResolvedSandbox> {
  if (mode === 'auto') {
    // Try: Seatbelt → Docker → Podman → Off
    for (const sb of [new SeatbeltSandbox(), new DockerSandbox(), new
      PodmanSandbox()]) {
      if (await sb.available()) return { sandbox: sb, effectiveMode:
        sb.mode };
    }
    return { sandbox: new NoSandbox(), effectiveMode: 'off' };
  }
  // Explicit mode selection...
}
```

Security System (src/security/)

Encryption Architecture



AES-256-GCM Encryption

Parameters

Parameter	Size	Description
Algorithm	—	AES-256-GCM
Key (DEK)	32 bytes	Data Encryption Key
IV	12 bytes (96 bits)	Random per encryption
Auth Tag	16 bytes	GCM authentication tag

EncryptedPayload

```

interface EncryptedPayload {
    iv: string;           // Base64-encoded 12-byte IV
    tag: string;          // Base64-encoded 16-byte GCM auth tag
  }
  
```

```
ciphertext: string; // Base64-encoded ciphertext
}
```

Functions

```
function generateDek(): Buffer; // Random 32 bytes
function encrypt(plaintext: string, dek: Buffer): EncryptedPayload;
function decrypt(payload: EncryptedPayload, dek: Buffer): string;
```

OS Keychain Integration

Keytar Wrapper

Uses the `keytar` native module for cross-platform keychain access:

Platform	Backend
macOS	Keychain Services
Linux	libsecret (GNOME Keyring / KWallet)
Windows	Windows Credential Manager

Per-Project Key Binding

```
function projectKeychainKey(projectRoot: string): string {
  const abs = path.resolve(projectRoot);
  const hash = createHash('sha256').update(abs).digest('hex');
  const user = os.userInfo().username || 'unknown';
  return `${user}:${hash.slice(0, 32)}`;
}
```

Account format: `<username>:<sha256(cwd)[..32]>`

This ensures each project gets its own DEK in the keychain, preventing cross-project key reuse.

Keychain Operations

```
async function getDek(projectRoot: string): Promise<Buffer | null>;
async function setDek(projectRoot: string, dek: Buffer): Promise<void>;
```

```
async function deleteDek(projectRoot: string): Promise<boolean>;  
function isAvailable(): boolean;
```

Test Mode

```
// Enable fake keychain for tests  
process.env.CODEXREV_TEST_FAKE_KEYCHAIN = '1';  
  
// Uses in-memory Map<string, string> instead of real keychain  
const fakeStore: Map<string, string> = new Map();
```

Error Hierarchy

```
CodexrevError (base)  
├─ ConfigError  
├─ AuthError  
├─ ToolError  
├─ SandboxError  
├─ McpError  
├─ CheckpointError  
├─ SecretsError  
└─ LocalServerError
```

MCP Integration (src/mcp/)

Model Context Protocol

MCP allows external tool servers to provide additional tools to the agent.

Transport Types

Transport	Protocol	Use Case
stdio	stdin/stdout	Local subprocesses
sse	Server-Sent Events	Remote HTTP servers
http	Streamable HTTP	Remote HTTP servers

Configuration

```
{
  "mcpServers": {
    "my-server": {
      "transport": "stdio",
      "command": "node",
      "args": ["server.js"],
      "env": { "API_KEY": "..."},
      "timeoutMs": 30000
    },
    "remote-server": {
      "transport": "http",
      "url": "https://example.com/mcp",
      "headers": { "Authorization": "Bearer ..." }
    }
  }
}
```

MCP Registry

```
class McpRegistry {
  async connect(name: string, entry: McpServerEntry): Promise<void>;
  async disconnectAll(): Promise<void>;
  async listToolDeclarations(): Promise<ToolDeclaration[]>;
}
```

```
    async callTool(name: string, args: unknown): Promise<ToolResult>;  
  }
```

Tool naming: MCP tools are prefixed with the server name:

`serverName__toolName`

Extension System (src/extensions/)

Extension Manifest

```
{
  "name": "my-extension",
  "version": "1.0.0",
  "description": "A custom extension",
  "hooks": ["onStart", "onToolCall"],
  "commands": [...],
  "tools": [...],
  "themes": [...],
  "prompts": [...],
  "mcp": [...]
}
```

Extension Loading

Extensions are scanned from `~/.codexrev/extensions/*/codexrev-extension.json`.

```
async function loadExtensions(opts?: { verbose?: boolean }):
  Promise<ExtensionRegistry> {
  const extensionsDir = getCodexrevPaths().extensionsDir;
  // Scan directories, validate manifests, build registry
}

async function installExtension(sourceDir: string): Promise<string>;
async function uninstallExtension(name: string): Promise<void>;
```

Checkpointing System (src/services/checkpoint.ts)

Shadow Git Repository

Each session gets its own shadow git repo at `~/.codexrev/checkpoints/<session-id>/`.

```
~/.codexrev/checkpoints/
├── <session-id>/
│   ├── .git/           Shadow git repo
│   ├── files/          Mirrored file contents
│   └── meta.json        Checkpoint metadata
```

Checkpoint Operations

Commit (Snapshot)

```
async function commit(label: string, files: ReadonlyArray<string>):
    Promise<CheckpointRecord>;
```

1. Validates files are within `rootDir`
2. Copies files to shadow repo's `files/` directory
3. Creates git commit with label
4. Records metadata in `meta.json`

Restore

```
async function restore(id: string): Promise<ReadonlyArray<string>>;
```

1. Looks up checkpoint metadata
2. Copies files back from shadow repo to working directory

List

```
async function list(): Promise<ReadonlyArray<CheckpointRecord>>;
```

Returns checkpoints ordered by creation time (newest first).

CheckpointRecord

```
interface CheckpointRecord {  
  readonly id: string;           // 12-char hex  
  readonly hash: string;         // Full git commit hash  
  readonly label: string;        // Human-readable description  
  readonly createdAt: string;    // ISO-8601 timestamp  
  readonly files: ReadonlyArray<string>; // Relative file paths  
}
```

Telemetry (src/telemetry/)

OpenTelemetry Integration

Built-in distributed tracing using the OpenTelemetry SDK with OTLP/gRPC export.

Functions

```
function initTelemetry(): void;
function shutdownTelemetry(): Promise<void>;
function withSpan<T>(name: string, fn: () => Promise<T>): Promise<T>;
function withSpanSync<T>(name: string, fn: () => T): T;
function recordEvent(name: string, attributes?: Record<string, unknown>): void;
```

Configuration

Variable	Default	Description
CODEXREV_OTEL_ENDPOINT	http://localhost:4317	OTLP/gRPC endpoint
CODEXREV_TELEMETRY	false	Enable/disable

Signal Handlers

Flushes telemetry data on `SIGTERM` and `SIGINT`.

Terminal UI (src/ui/)

React + Ink Architecture

The TUI is built with **React** and **Ink** (React renderer for terminal).

Main Component (App.tsx)

State: - `messages` : Conversation history - `input` : Current user input -
- `busy` : Whether the agent is processing - `mode` : Current interaction
mode - `clarificationRequest` : Pending clarification -
- `fixConfirmRequest` : Pending fix confirmation

Slash Commands:

Command	Description
<code>/help</code>	Show help text
<code>/clear</code>	Clear conversation
<code>/tools</code>	List available tools
<code>/quit</code> , <code>/exit</code>	Exit the application
<code>/theme</code>	Toggle theme
<code>/mode</code>	Cycle mode (Ask → Plan → Agent)

Tab Key: Cycles through modes (Ask → Plan → Agent)

Mode Badge

Color-coded badge shown next to the input:

- **Ask:** Blue badge
- **Plan:** Yellow badge
- **Agent:** Green badge

Clarification Prompt

Renders clarification questions with:

- Suggestion buttons (when provided)
- Free-form text input
- Shows question text from the pipeline

Fix Confirm Prompt

Shows verification failure status and offers:

- **Continue:** Retry with error context
- **Stop:** Keep current state

Init Wizard (5 Steps)

1. **Provider:** Select LLM provider
2. **Model:** Select model
3. **API Key:** Enter and encrypt API key
4. **Base URL:** Optional custom endpoint
5. **Confirm:** Review and save

Models Wizard (11+ Steps)

Multi-step wizard for adding providers and models with validation for each field.

Public API (src/api/index.ts)

The package exposes a programmatic API for embedding Codexrev in other applications.

Exports

```
// Agent
export { runAgent, type AgentEvent, type AgentResult, type
  RunAgentOptions } from '../core/turn.js';
export { createAgent } from '../core/agent.js';

// Pipeline
export { runPipeline, type PipelineResult } from '../core/pipeline.js';
export { runFixLoop, type FixLoopResult } from '../core/fixLoop.js';

// Providers
export { buildProvider } from '../providers/index.js';
export { PROVIDER_REGISTRY, PROVIDER_IDS, providerMeta } from '../
  providers/registry.js';

// Config
export { loadSettings, saveSettings } from '../config/loader.js';
export { DEFAULT_SETTINGS, type Settings } from '../config/schema.js';

// Tools
export { builtinTools } from '../tools/builtin.js';
export { ToolRegistry, createToolRegistry } from '../tools/registry.js';

// MCP
export { McpRegistry } from '../mcp/registry.js';

// Security
export { encrypt, decrypt, generateDek } from '../security/secrets.js';
export { getDek, setDek, deleteDek, isAvailable } from '../security/
  keychain.js';

// Checkpointing
export { CheckpointService, type CheckpointRecord } from '../services/
  checkpoint.js';

// Sandbox
export { resolveSandbox } from '../sandbox/manager.js';

// Telemetry
export { initTelemetry, shutdownTelemetry } from '../telemetry/index.js';
```

```
// Errors
export * from '../utils/errors.js';

// Types
export type { ProviderId, Message, ContentGenerator, StreamEvent } from
  '../core/types.js';
export type { InteractionMode } from '../core/modes.js';
```

Usage Example

```
import { createAgent } from 'codexrev';

const agent = createAgent({
  provider: 'openai',
  model: 'gpt-4o',
  apiKey: process.env.OPENAI_API_KEY,
});

// Non-streaming
const result = await agent.send('Explain this code');
console.log(result.finalText);

// Streaming
for await (const event of agent.stream('Fix the bug')) {
  if (event.kind === 'text_delta') process.stdout.write(event.text);
}

agent.close();
```

Build System

esbuild Configuration

Five bundles are produced:

Output	Format	Description
dist/cli.js	ESM	CLI entry with shebang
dist/api.js	ESM	Programmatic API
dist/api.cjs	CJS	Programmatic API (CommonJS)
dist/tools.js	ESM	Tools-only entry
dist/tools.cjs	CJS	Tools-only entry

Externals (Not Bundled)

- LLM SDKs: `openai`, `@anthropic-ai/sdk`, `@google/genai`
- MCP: `@modelcontextprotocol/sdk`
- OpenTelemetry stack
- Ink/React/Yoga (top-level await + WASM)
- Native modules: `keytar`, `fsevents`

Post-Build Steps

1. Copy `assets/` directory
2. Write `version.json`
3. Prepend Node.js shebang to CLI
4. Set executable permissions (`chmod 755`)

TypeScript Configuration

```
{  
  "compilerOptions": {  
    "target": "ES2023",  
    "module": "NodeNext",
```

```

    "moduleResolution": "NodeNext",
    "jsx": "react-jsx",
    "strict": true,
    "declaration": true,
    "sourceMap": true,
    "outDir": "dist",
    "rootDir": "src"
  }
}

```

npm Scripts

Script	Command	Description
build	node scripts/build.js	Production build
start	node scripts/start.js	Run built CLI
dev	node scripts/start.js --dev	Development mode (tsx)
test	vitest run	Run tests
test:watch	vitest	Watch mode tests
lint	eslint .	Lint code
lint:fix	eslint . --fix	Auto-fix lint issues
format	prettier --write .	Format code
typecheck	tsc --noEmit	Type checking
clean	node scripts/clean.js	Remove build artifacts
release	npm publish --access public	Publish to npm

Dockerfile

Multi-stage Alpine build:

```

FROM node:20-alpine AS builder
# ... build steps ...

FROM node:20-alpine
RUN apk add --no-cache tini
RUN addgroup -S codexrev && adduser -S codexrev -G codexrev
USER codexrev
ENTRYPOINT ["/sbin/tini", "--"]
CMD ["node", "/app/dist/cli.js"]

```

Test Suite

Test Files

Test File	Coverage Area
tests/cli/init.test.ts	Non-interactive init, reset, local provider probes
tests/config/loader.test.ts	Layered merge, DEK decryption, tampered ciphertext
tests/config/models.test.ts	Add/remove providers/models, set default
tests/config/projectConfig.test.ts	Paths, round-trip, schema version
tests/providers/health.test.ts	resolveBaseUrl , probeLocalProvider
tests/providers/registry.test.ts	Metadata validation, type guard
tests/security/keychain.test.ts	Fake mode, per-project binding
tests/security/secrets.test.ts	Round-trip, wrong key, tampered ciphertext, wrong-size DEK

Key Test Patterns

Keychain Tests (Fake Mode)

```
beforeEach(() => {
  process.env.CODEXREV_TEST_FAKE_KEYCHAIN = '1';
  _resetFakeStore();
});
```

Secrets Round-Trip

```
it('encrypts and decrypts correctly', () => {
  const dek = generateDek();
  const payload = encrypt('hello world', dek);
```

```
    const result = decrypt(payload, dek);
    expect(result).toBe('hello world');
  });

it('rejects wrong key', () => {
  const dek1 = generateDek();
  const dek2 = generateDek();
  const payload = encrypt('hello', dek1);
  expect(() => decrypt(payload, dek2)).toThrow();
});
```

Settings Merge

```
it('applies env overrides', () => {
  process.env.CODEXREV_PROVIDER = 'anthropic';
  process.env.CODEXREV_MODEL = 'claude-3-5-sonnet-20241022';
  const settings = loadSettings();
  expect(settings.provider).toBe('anthropic');
  expect(settings.model).toBe('claude-3-5-sonnet-20241022');
});
```

Error Handling

Error Hierarchy

CodexrevError

- └─ ConfigError Configuration loading/validation failures
- └─ AuthError Authentication/authorization failures
- └─ ToolError Tool execution failures
- └─ SandboxError Sandbox execution failures
- └─ McpError MCP server connection failures
- └─ CheckpointError Checkpoint service failures
- └─ SecretsError Encryption/decryption failures
- └─ LocalServerError Local provider connection failures

Provider Error

```
class ProviderError extends Error {
  constructor(
    message: string,
    public readonly provider: ProviderId,
    public readonly httpStatus?: number,
    public readonly code?: string,
    public readonly retryable: boolean = false,
  ) { ... }
}
```

Error Recovery

Error Type	Recovery Strategy
ProviderError (retryable)	Automatic retry with backoff
LocalServerError	Fail fast with friendly hint
ConfigError	Exit with setup instructions
SandboxError	Fall back to NoSandbox
ToolError	Report error to model, continue
SecretsError	Prompt user to re-run <code>init --reset</code>

Configuration Files Reference

File Locations

File	Location	Purpose
User settings	~/.codexrev/settings.json	User-level preferences
Project settings	.codexrev/settings.json	Project-level settings
Project config	.codexrev/config.json	Encrypted API keys
Extensions	~/.codexrev/extensions/*/	Installed extensions
Checkpoints	~/.codexrev/checkpoints/ <session>/	Session snapshots
Logs	stderr (JSON)	Structured logging

Directory Permissions

Path	Permission	Description
.codexrev/	0700	Project config directory
.codexrev/config.json	0600	Encrypted config file

Atomic Writes

All configuration writes use the **tmp + rename** pattern:

```
const tmp = `${file}.tmp.${process.pid}`;  
await fs.writeFile(tmp, content, { mode: 0o600 });  
await fs.rename(tmp, file);
```

Environment Variables Reference

Provider Keys

Variable	Provider	Description
OPENAI_API_KEY	OpenAI	API key for OpenAI
OPENAI_BASE_URL	OpenAI	Custom base URL
ANTHROPIC_API_KEY	Anthropic	API key for Anthropic
ANTHROPIC_BASE_URL	Anthropic	Custom base URL
GOOGLE_API_KEY	Google	API key for Google Gemini
OLLAMA_API_KEY	Ollama	API key (sentinel)
OLLAMA_BASE_URL	Ollama	Ollama server URL
LMSTUDIO_API_KEY	LM Studio	API key (sentinel)
LMSTUDIO_BASE_URL	LM Studio	LM Studio server URL
LITELLM_API_KEY	LiteLLM	API key
LITELLM_BASE_URL	LiteLLM	LiteLLM proxy URL

Settings Overrides

Variable	Description
CODEXREV_PROVIDER	Override active provider
CODEXREV_MODEL	Override active model
CODEXREV_MAX_TOKENS	Override max output tokens
CODEXREV_TEMPERATURE	Override temperature
CODEXREV_THEME	Override UI theme
CODEXREV_SANDBOX	Override sandbox mode
CODEXREV_TELEMETRY	Enable telemetry
CODEXREV_CHECKPOINTING	Enable checkpointing
CODEXREV_DEFAULT_MODE	Set default mode
CODEXREV_MAX_FIX_ATTEMPTS	Set max fix attempts
CODEXREV_VERIFICATION_MODE	Set verification strategy

Internal

Variable	Description
<code>CODEXREV_LOG_LEVEL</code>	Log level: debug/info/warn/error/silent
<code>CODEXREV_OTEL_ENDPOINT</code>	OpenTelemetry endpoint
<code>CODEXREV_TEST_FAKE_KEYCHAIN</code>	Use in-memory keychain

Appendix A: Provider Implementation Details

OpenAI Provider

- **SDK:** `openai` npm package
- **Streaming:** NDJSON chunks via `response.body`
- **Tool calls:** Accumulated across chunks by `index` field
- **Finish reasons:** `stop`, `max_tokens`, `tool_use`, `error`

Anthropic Provider

- **SDK:** `@anthropic-ai/sdk`
- **Streaming:** SSE events (`content_block_start`, `delta`, `stop`)
- **Tool calls:** Accumulated via `input_json_delta`
- **Cache tracking:** `cache_read_input_tokens`, `cache_creation_input_tokens`

Google Gemini Provider

- **SDK:** `@google/genai`
- **Streaming:** Async iterator of chunks
- **Tool calls:** `functionCall` parts in response
- **Content format:** Google's `Content` type with `parts[]`

Ollama Provider

- **API:** Hand-rolled fetch to `/api/chat` (OpenAI compat also available)
- **Streaming:** NDJSON
- **No API key required:** Uses sentinel `'ollama'`

- **No retries:** `maxRetries = 0`

LM Studio Provider

- **Base:** Thin subclass of OpenAI-compatible
- **Default URL:** `http://localhost:1234/v1`
- **No API key required:** Uses sentinel `'lmstudio'`

LiteLLM Provider

- **Base:** Thin subclass of OpenAI-compatible
- **Default URL:** `http://localhost:4000`
- **Use case:** Proxy for multiple providers

Appendix B: Tool Parameter Schemas

read_file

```
{
  "type": "object",
  "properties": {
    "file_path": { "type": "string", "description": "Absolute path to the file" },
    "offset": { "type": "number", "description": "Line offset (0-based)" },
    "limit": { "type": "number", "description": "Max lines to read" }
  },
  "required": ["file_path"]
}
```

write_file

```
{
  "type": "object",
  "properties": {
    "file_path": { "type": "string", "description": "Absolute path to write" },
    "content": { "type": "string", "description": "File content" }
  },
  "required": ["file_path", "content"]
}
```

edit

```
{
  "type": "object",
  "properties": {
    "file_path": { "type": "string", "description": "Absolute path to the file" },
    "old_string": { "type": "string", "description": "Exact string to find" },
    "new_string": { "type": "string", "description": "Replacement string" }
  }
}
```

```

    "replace_all": { "type": "boolean", "description": "Replace all
                     occurrences" },
  },
  "required": ["file_path", "old_string", "new_string"]
}

```

shell

```

{
  "type": "object",
  "properties": {
    "command": { "type": "string", "description": "Shell command" },
    "timeout": { "type": "number", "description": "Timeout in ms" },
  },
  "required": ["command"]
}

```

glob

```

{
  "type": "object",
  "properties": {
    "pattern": { "type": "string", "description": "Glob pattern" },
    "cwd": { "type": "string", "description": "Working directory" },
  },
  "required": ["pattern"]
}

```

grep

```

{
  "type": "object",
  "properties": {
    "pattern": { "type": "string", "description": "Regex pattern" },
    "path": { "type": "string", "description": "Directory to search" },
    "include": { "type": "string", "description": "File glob filter" },
    "case_sensitive": { "type": "boolean", "description": "Case
                     sensitivity" },
  },
  "required": ["pattern", "path"]
}

```

web_fetch

```
{
  "type": "object",
  "properties": {
    "url": { "type": "string", "description": "HTTP/HTTPS URL" }
  },
  "required": ["url"]
}
```

web_search

```
{
  "type": "object",
  "properties": {
    "query": { "type": "string", "description": "Search query" },
    "num_results": { "type": "number", "description": "Number of
                    results" }
  },
  "required": ["query"]
}
```

Appendix C: Glossary

Term	Definition
DEK	Data Encryption Key — 32-byte key used for AES-256-GCM encryption
SBPL	Seatbelt Profile Language — macOS sandbox configuration format
MCP	Model Context Protocol — standard for external tool servers
NDJSON	Newline-Delimited JSON — streaming format used by OpenAI-compatible APIs
GCM	Galois/Counter Mode — authenticated encryption mode for AES
IV	Initialization Vector — random value used in encryption
Shadow Git	Isolated git repository used for checkpointing without affecting the user's repo
Fix Loop	Automatic retry mechanism when verification fails
Committee	Pipeline phase for read-only analysis
Breaker-Builder	Pipeline phase for implementation
Resolver	Pipeline phase for verification
InteractionChannel	Async producer-consumer for mid-run user Q&A
ContentGenerator	Unified interface for all LLM providers
ToolDeclaration	JSON Schema describing a tool's parameters
ToolContext	Runtime context passed to tool execution (cwd, signal)

Appendix D: Version History

Version	Date	Changes
0.1.0	2026-08-15	Initial release

Roadmap Items

- Public demo hosting
 - Self-hosted eval suite with real LLM
 - Multi-turn session resume polish (`--resume` flag)
 - Extension marketplace
 - Custom tool definitions via config
 - Theme customization API
 - Performance profiling dashboard
-

This document was generated from the Codexrev CLI source code at version 0.1.0. Last updated: August 2026